M2 → M4 개인화 학습 피드백 필드 추가 (code_context / taxonomy)

날짜: 2026-06-04 범위: Dev2(M2) 코드 리뷰 모듈 — `schema.py`, `nodes/concept_mapper.py`, `nodes/store.py` 목적: M4(개인화 학습 피드백)가 학습자료를 생성할 때 쓸 구조화 정보를 M2 출력의 각 이슈에 최소 추가한다. M2는 학습자료를 직접 만들지 않고, M4가 소비할 데이터만 채운다.

---

1. 추가한 필드 2개

각 `issue` 객체에 아래 두 필드를 추가했다. (기존 필드는 하나도 제거하지 않음)

1-A. `code_context` — 이슈 라인 주변 코드 맥락

```
{
  "code_context": {
    "context_before": ["def get_user(request):", "    username = reques
    "issue_line": "    query = \"SELECT * FROM auth_user WHERE username
    "context_after": ["    users = User.objects.raw(query)"],
    "context_summary": "사용자 입력값이 SQL 문자열에 직접 결합되어 실행되는 코
  }
}
```

생성 규칙:

- `pr_diff` / `file` / `line` / `code_snippet`만 사용 — 새 **LLM** 호출 없음, 전부 규칙 기반.
- unified diff를 파싱해 이슈 라인 앞뒤 3줄 추출(들여쓰기 보존).
- diff에서 라인을 못 찾으면 `issue_line = code_snippet`.
- `code_snippet`도 없으면 `"snippet_unavailable"`.
- `context_summary`는 concept/message 기반 규칙으로 생성하며 절대 빈 문자열이 아니다.

1-B. `taxonomy` — 보안/정적분석 하위 분류

```
{
  "taxonomy": {
    "category": "Security",
    "subcategory": "Injection",
    "issue_type": "SQL Injection",
    "sub_issue_type": "Raw Query String Formatting"
  }
}
```

분류 규칙(위→아래 우선순위, 먼저 매칭되는 규칙 적용):

조건	**category / subcategory / issue_type / sub_issue_type**

`rule_id == "B608"` 또는 `mistake_type == "raw_query"`	Security / Injection / SQL Injection / Raw Query String Formatting
`mistake_type == "no_parameterize"`	Security / Injection / SQL Injection / Missing Parameterized Query
`mark_safe` 감지 또는 `mistake_type == "mark_safe_misuse"`	Security / XSS / Cross-Site Scripting / mark_safe Misuse
`|safe` 감지 또는 `mistake_type == "safe_filter_misuse"`	Security / XSS / Cross-Site Scripting / Template Safe Filter Misuse
`innerHTML` 감지 또는 `mistake_type == "inner_html_direct"`	Security / XSS / Cross-Site Scripting / Direct innerHTML Assignment
`csrf_exempt` 감지 또는 `mistake_type == "csrf_exempt_misuse"`	Security / CSRF / Cross-Site Request Forgery / csrf_exempt Misuse
`mistake_type == "no_csrf_token"`	Security / CSRF / Cross-Site Request Forgery / Missing CSRF Token
`mistake_type == "no_login_required"`	Security / Authentication & Authorization / Missing Authentication Check / Missing login_required
`mistake_type == "no_permission_check"`	Security / Authentication & Authorization / Authorization Bypass / Missing Object Permission Check
Radon / complexity 관련	Code Quality / Complexity / Cyclomatic Complexity / High Cyclomatic Complexity
일반 Pylint/ESLint lint	Code Quality / Maintainability / Code Smell / General Code Smell
fallback	General / Code Quality / Unknown / General Review Issue

> 성능 / 테스트 / 함수 정밀 분석은 M3 담당이므로 taxonomy에서 분류하지 않는다.

---

2. 변경 파일

파일	변경 내용
`dev2/schema.py`	`CodeContext`, `IssueTaxonomy` Pydantic 모델 추가 + `Issue`에 `code_context`, `taxonomy` 필드 추가
`dev2/nodes/concept_mapper.py`	`build_code_context()`, `build_taxonomy()` + 헬퍼 추가, `enrich_with_concept_mapping()`에 통합
`dev2/nodes/store.py`	`_ISSUE_FIELDS` 화이트리스트에 `code_context`, `taxonomy` 추가 (Pydantic 경계 통과용)
`dev2/tests/test_concept_mapper.py`	테스트 21건 추가 (총 52건)

---

3. M4가 받는 결과물 구조

M4는 M2가 `store_node`에서 반환하는 **`final_output`**(CLAUDE.md §5-A Output Contract,

AgentOutput.model_dump()의 리스트)을 받는다. 추가한 두 필드는 최상위가 아니라 **issues[]** 안의 각 이슈에 붙는다.

```
[
  {
    "agent": "security_scan",
    "task_id": "uuid",
    "issues": [
      {
        // ── 기존 Output Contract 필드 ──
        "severity": "Critical",
        "issue_type": "Vulnerability",
        "line": 3,
        "file": "src/views.py",
        "message": "SQL 인젝션 가능성",
        "rule_id": "B608",
        "suggestion": "파라미터화된 쿼리 사용",
        "resolution_example": "",
        "status": "Open",
        "similar_issues": [],

        // ── 기존 M4 매핑 필드 ──
        "concept": "SQL Injection",
        "mistake_type": "raw_query",
        "concept_id": "sql_injection",
        "mistake_type_concept_id": "sql_injection.raw_query",
        "confidence_score": 0.92,
        "code_snippet": "query = \"SELECT ...\" % username",
        "learning_context": { "framework": "django", "language": "pytho

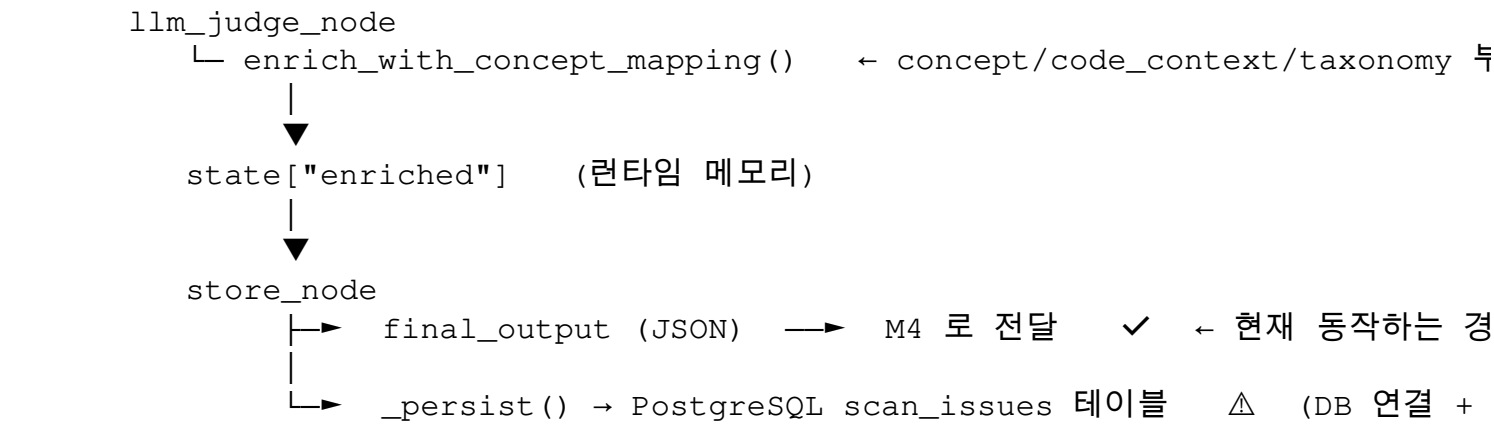
        // ── 신규 M4 피드백 필드 ──
        "code_context": {
          "context_before": ["def get_user(request):", "    username =
          "issue_line": "    query = \"SELECT * FROM auth_user WHERE us
          "context_after": ["    users = User.objects.raw(query)"],
          "context_summary": "사용자 입력값이 SQL 문자열에 직접 결합되어 실행!
        },
        "taxonomy": {
          "category": "Security",
          "subcategory": "Injection",
          "issue_type": "SQL Injection",
          "sub_issue_type": "Raw Query String Formatting"
        }
      }
    ],
    "summary": { "critical": 1, "major": 0, "minor": 0, "recommended_ac
    "execution_ms": 1200,
    "metadata": { "action_decision": { } }
  }
]
```

M4 입장에서는 기존 **Output Contract**를 그대로 받되, 각 이슈에 **code_context·taxonomy**가 추가로 딸려오는 형태다. "필드 추가 가능, 삭제 불가" 계약을 지켜서 기존 소비자 로직은 영

향받지 않는다.

---

4. 데이터 흐름



호출 지점은 기존 파이프라인 그대로다. `llm_judge_node`가 severity를 재판단한 뒤 호출하는 `enrich_with_concept_mapping`에서 두 필드를 함께 채운다.

---

5. DB 저장 관련 메모

현재 두 필드(및 기존 M4 매핑 필드 전체)는 **PostgreSQL** 테이블에는 저장되지 않는다. `final_output` JSON으로 M4에 전달되는 것이 전부이며, 이는 원래 작업 명세("store 직전 또는 final_output 조립 시 붙여도 됨")를 충족한다.

세 테이블의 역할(저장 단위):

테이블	함수	저장 단위	내용
`scan_executions`	`insert_scan_execution`	실행 1회 → `scan_id` 발급	task_id, pr_id, developer_id, execution_ms, trace_id
`scan_issues`	`insert_scan_issues`	이슈 **1**개 = **1** 행	severity, rule_id, file, line, message, suggestion, similar_issues
`scan_summaries`	`insert_scan_summary`	agent별 집계 1행	critical / major / minor 개수만

→ 이슈 단위 데이터이므로, 저장한다면 **`scan_issues`** 가 맞다. `scan_summaries`는 개수 집계 전용이라 해당 없음.

테이블 저장이 성립하려면 아래 3가지가 모두 필요하다(현재 미충족):

1. 실제 Cloud SQL 인스턴스 + `DATABASE_URL` 설정 (db.py:16)
2. `scan_issues` 테이블에 컬럼 추가 — 예: `ALTER TABLE scan_issues ADD COLUMN m4_metadata JSONB;`
3. `insert_scan_issues` INSERT문에 해당 필드 추가 (현재 9개 컬럼 고정)

> 권장: `similar_issues`가 이미 JSON으로 저장되는 패턴과 동일하게, M4 필드를 `m4_metadata JSONB` 단일 컬럼에 묶어 저장하는 방식.

6. 테스트

```
cd sub2_new
python -m pytest dev2/tests/test_concept_mapper.py -q          # 52 pas
python -m pytest dev2/tests/test_schema.py dev2/tests/test_nodes.py dev
```

검증 항목:

- B608 → SQL Injection taxonomy 생성
- code_snippet fallback 동작 (diff 없음 → `"snippet_unavailable"`)
- `context_summary`가 빈 문자열이 아님
- 기존 output 필드 무손실 (concept/mistake_type/learning_context 등 유지)
- `code_context·taxonomy` dict이 `Issue` Pydantic 경계 통과

  DB 없이 통과한다 — 테스트는 `patch("dev2.db.AsyncSessionLocal")`로 DB를 Mock 처리하며, 추가 필드 로직 자체는 DB를 건드리지 않는다.